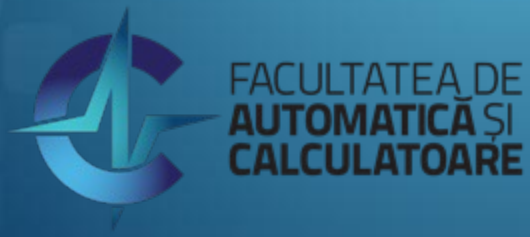


Algoritmi Paraleli și Distribuiți
Mecanisme de sincronizare

Radu-Ioan CIOBANU
radu.ciobanu@upb.ro





Race Conditions

Process a:

```
while(i < 10)
    i = i + 1;
print "A won!";
```

Process b:

```
while(i > -10)
    i = i - 1;
print "B won!";
```



Problema secțiunilor critice

- Problema: Dezvoltarea unui protocol pentru cooperare între procese, a.î. execuția să aibă loc mutual exclusiv (secțiuni critice)
  - Cum să facem ca mai multe instrucțiuni să pară ca una singură?

Shared vars:

Initialization:

Process:
. . .
. . .

Entry Section

Critical Section

Exit Section

Soluția constă în introducerea acestor mecanisme

Cerințe ale soluției

Discuție asupra folosirii semafoarelor pentru soluționare...

- **Mutual Exclusion**
  - Doar un proces poate fi la un moment dat în secțiunea critică
- **Progress**
  - Decizia cine intră în regiune nu poate fi amânată indefinit
    - No deadlock
- **Bounded Waiting**
  - Limită asupra #times alte procese pot intra în regiune, în timp ce aștept
    - No livelock or starvation
- La care se adaugă eficiență, fairness …

Reminder: Dekker's Algorithm

- Presupune două thread-uri, numerotate cu 0 și 1

```
variables
   wants_to_enter : array of 2 booleans
   turn : integer
wants_to_enter[0] ← false
wants_to_enter[1] ← false
turn ← 0 // or 1
```

```
p0:
   wants_to_enter[0] ← true
   while (wants_to_enter[1]) {
      if (turn ≠ 0) {
         wants_to_enter[0] ← false
         while (turn ≠ 0) { // busy wait }
         wants_to_enter[0] ← true
      }
   }
   // critical section ...
   turn ← 1
   wants_to_enter[0] ← false
```

```
p1:
   wants_to_enter[1] ← true
   while (wants_to_enter[0]) {
      if (turn ≠ 1) {
         wants_to_enter[1] ← false
         while (turn ≠ 1) { // busy wait }
         wants_to_enter[1] ← true
      }
   }
   // critical section ...
   turn ← 0
   wants_to_enter[1] ← false
```

Peterson's Algorithm (1981)

```
CSEnter(int i) {
        inside[i] = true;
        turn = j;
        while (inside[j] && turn == j)
                continue;
}
```

```
CSExit(int i) {
  inside[i] = false;
  }
```

- Mai simplu e mai bine!!

O analiză a algoritmului lui Peterson:

- Safety (prin contradicție):
  - Presupuneți că două procese (Ana și Ion) sunt în regiunea critică (ambele au flag-ul *inside* setat). Deoarece doar unul, să spunem Ana, poate avea *turn*, celălalt (Ion) **trebuie** să fi ajuns la testul *while()* înainte ca Ana să seteze flag-ul *inside.*
  - Totuși, după setarea flag-ului *inside*, Ana a cedat *turn* către Ion. Ion deja a modificat *turn* și **nu poate** să îl modifice din nou, ceea ce contravine presupunerii inițiale.

Liveness & Bounded waiting => date de variabila *turn*.

Putem generaliza pentru mai multe thread-uri?

- Abordarea anterioară nu funcționează:

```
CSEnter(int i) {
        inside[i] = true;
        for (j = 0; j < N; j++)
                while (inside[j] && turn == j)
                        continue;
}
```

```
CSExit(int i) {
        inside[i] = false;
}
```



Conceptul de Bakery

- Introdus de Leslie Lamport
- Gândiți-vă la un magazin popular cu un vânzător (ex. cazul unui Apple Store și lansarea următorului iPhone)
  - Oamenii iau un bilet de la un automat
  - Dacă nimeni nu așteaptă, biletul nu mai contează
  - Dacă mai mulți clienți așteaptă, ordinea biletelor determină ordinea în care vor fi serviți



Algoritmul Bakery : “Runda 1”

- int ticket[n];
- int next_ticket;

```
CSEnter(int i) {
        ticket[i] = ++next_ticket;
        for (j = 0; j < N; j++)
                while(ticket[j] && ticket[j] < ticket[i])
                        continue;
}
```

```
CSExit(int i) {
        ticket[i] = 0;
}
```



Algoritmul Bakery: “Runda 2”

- int ticket[n];

Adăugăm 1 la maxim!

```
CSEnter(int i) {
        ticket[i] = max(ticket[0], … ticket[N-1])+1;
        for (j = 0; j < N; j++)
                while (ticket[j] && ticket[j] < ticket[i])
                        continue;
}
```

```
CSExit(int i) {
        ticket[i] = 0;
}
```

- Idee: adăugăm 1 la maxim.
- Oops…



Algoritmul Bakery: “Runda 3”

Dacă i și j aleg un același bilet, stabilim o regulă tip “break tie”:

(ticket[j] < ticket[i]) || (ticket[j]==ticket[i] && j<i)

Notație: (B,j) < (A,i) pentru a simplifica scrierea codului:

(B<A || (B==A && j<i)), e.g.:

(ticket[j],j) < (ticket[i],i)

Algoritmul Bakery: “Runda 4”

- int ticket[N];
- boolean picking[N] = false;

```
CSEnter(int i)
{
    ticket[i] = max(ticket[0], … ticket[N-1])+1;
    for (j = 0; j < N; j++)
        while(ticket[j] && (ticket[j], j) < (ticket[i],i))
            continue;
}
```

```
CSExit(int i)
{
    ticket[i] = 0;
}
```

- Oops…



Algoritmul Bakery: Aproape gata

- int ticket[N];
- boolean choosing[N] = false;

```
CSEnter(int i)
{
    choosing[i] = true;
    ticket[i] = max(ticket[0], … ticket[N-1])+1;
    choosing[i] = false;
    for (j = 0; j < N; j++) {
        while(choosing[j]) continue;
        while(ticket[j] && (ticket[j], j) < (ticket[i],i))
            continue;
    }
}
```

```
CSExit(int i)
{
    ticket[i] = 0;
}
```

Algoritmul Bakery: Probleme?

- Dacă nu știm câte thread-uri rulează?
  - Algoritmul depinde de valoarea N
  - Cumva avem nevoie de un mecanism de ajustare a N atunci când un thread e creat sau termină execuția
- De asemenea, tehnic, un bilet poate da overflow!
  - Soluția: Modificarea codului ca, atunci când un bilet ajunge la o valoare prea mare, să îl resetăm la 0

Algoritmul Bakery: Final

- int ticket[N]; /* Important: Disable thread scheduling when changing N */
- boolean choosing[N] = false;

```
CSEnter(int i)
{
        do {
            ticket[i] = 0;
            choosing[i] = true;
            ticket[i] = max(ticket[0], … ticket[N-1])+1;
            choosing[i] = false;
        } while(ticket[i] >= MAXIMUM);
        for (j = 0; j < N; j++) {
                while(choosing[J]) continue;
                while(ticket[J] && (ticket[J],J) < (ticket[i],i))
                        continue;
        }
}
```

```
CSExit(int i)
{
        ticket[i] = 0;
}
```

Ne reîntoarcem la semafoare

- Unele sisteme (acolo unde busy-waiting nu e o problemă, ex. există suficiente core-uri) implementează soluții bazate pe Algoritmul Bakery
- Alteori (cel mai des însă) avem nevoie de *semafor*
  - Soluție cu *blocare* în loc de *busy-waiting* de la SO

Blocare vs. non-blocare

```
acquire() {
    while (value <= 0)
        ; // no-op
    value--;
}

release() {
    value++;
}
```

```
acquire() {
    value--;
    if (value < 0) {
        add this process to list
        block();
    }
}

release() {
    value++;
    if (value <= 0) {
        remove a process P from list
        wakeup(P);
    }
}
```

Un pic de recap... ce e greșit?

Shared: Semaphores mutex, empty, full;

Init: mutex = 1; /* for mutual exclusion*/
empty = N; /* number empty bufs */
full = 0; /* number full bufs */

Producer

```
do {
    . . .
    // produce an item in nextp
    . . .
    P(mutex);
    P(empty);
    . . .
    // add nextp to buf
    . . .
    V(mutex);
    V(full);
} while (true);
```

Ce se întâmplă dacă buffer = full?



Consumer

```
do {
    P(full);
    P(mutex);
    . . .
    // remove item to nextc
    . . .
    V(mutex);
    V(empty);
    . . .
    // consume item in nextc
    . . .
} while (true);
```

Exercițiu

- Cum ati implementa o *barieră* folosind semafoare?

```
process P[k=1 to n]{
        ...
        barrier
    }
}
```



Posibila solutie… pentru bariera

```
sem b = 0; e = 1;
int nb = 0;
process proc[k=1 to n]{
    # enter barrier
    P(e);
    nb = nb + 1;
    if (nb < n){ V(e); P(b); }
    nb = nb - 1;
    if (nb > 0) V(b);
    elsif (nb = 0) V(e);
   # exit barrier
}
```

except last process to enter barrier

if first process to enter barrier

Un pic de practice: Cigarette smokers problem

- Un agent și trei fumători
- Fumătorii:
  - Așteaptă ingrediente (tutun, hârtie, chibrit)
  - Confecționează țigară
  - Fumează
- Agentul deține toate 3 ingredientele
- Un fumător are tutun, un altul hârtie, al 3-lea chibrituri
- Agentul selectează două ingrediente (random) pe care le dă fumătorilor
  - Doar fumătorul ce are nevoie de exact acele 2 ingrediente trebuie să le preia
  - Agentul nu poate semnaliza exact acelui fumător pentru că nu știe care fumător e care, respectiv ingredientele sunt random extrase

Cigarette smokers problem

```
sem tobacco = 0;
sem paper = 0;
sem match = 0;
sem agent = 1;
process Agent{
    while (true){
      if (draw1){ P(agent); V(tobacco); V(paper); }
      else if (draw2){ P(agent); V(paper); V(match); }
      else if (draw3){ P(agent); V(tobacco); V(match);}
    }
}
process Smoker1{
    P(tobacco); P(paper); V(agent);
}
process Smoker2{
    P(paper); P(match); V(agent);
}
process Smoker3{
    P(tobacco); P(match); V(agent);
}
```



Cigarette smokers problem - deadlock



OK!

P(match)

P(tobacco)



DEADLOCK!

P(paper)

P(match)



P(tobacco) P(paper)



Practice

- Gandiți-vă la o soluție pentru evitarea deadlock-ului…



Cigarette smokers problem

```
sem tobacco = 0;
sem paper = 0;
sem match = 0;
sem agent = 1;

bool isTobacco = false;
bool isPaper = false;
bool isMatch = false;

sem tobaccoSem = 0;
sem paperSem = 0;
sem matchSem = 0;

process Agent{
    while (true) {
      if (draw1) { P(agent); V(tobacco); V(paper);}
      else if (draw2) { P(agent); V(paper); V(match);}
      else if (draw3) { P(agent); V(tobacco); V(match);}
    }
}
```

Cigarette smokers problem

```
process PusherA{
     P(tobacco);
     P(e);
     if (isPaper) { isPaper =  false; V(matchSem);}
     else if (isMatch) { isMatch = false; V(paperSem);}
     else if (isPaper == isMatch == false) isTobacco = true;
     V(e);
}

proces PusherB{
     P(match);
     P(e);
     if (isPaper) { isPaper =  false; V(tobaccoSem); }
     else if (isTobacco) { isTobacco = false; V(paperSem);}
     else if (isPaper == isTobacco == false) isMatch = true;
     V(e);
}

process PusherC{
     P(paper);
     P(e);
     if (isTobacco) { isTobacco =  false; V(matchSem);}
     else if (isMatch) { isMatch = false; V(tobaccoSem);}
     else if (isTobacco == isMatch == false) isPaper = true;
     V(e);
}
```

Cigarette smokers problem

```
process SmokerWithTobacco{
     P(tobaccoSem);
     # makeCigarette
     V(agent);
     # smoke
}
process SmokerWithPaper{
     P(paperSem);
     # makeCigarette
     V(agent);
     # smoke
}
process SmokerWithMatch{
     P(matchSem);
     # makeCigarette
     V(agent);
     # smoke
}
```

O altă problemă - East-West Bridge Problem

- Considerați un pod, prea îngust pentru a permite mașinilor să meargă în ambele direcții.
  - Mașinile trebuie să alterneze pt. trecerea podului.
  - Podul nu este, de asemenea, suficient de puternic pentru a ține mai mult de **trei** mașini simultan.
- Găsiți o soluție la această problemă care să evite **starvation**.
  - Mașinile care doresc să treacă ar trebui să ajungă în cele din urmă.
- Încercați să maximizați capacitatea podului (adică trei mașini ar trebui să meargă la un moment dat).
  - Dacă o mașină părăsește podul care se îndreaptă spre est și nu există mașini spre vest, atunci următoarea mașină spre est trebuie lăsată să treacă.


10
M.P.H.
alamy stock photo
DYC3N7
www.alamy.com

Soluția problemei

- Soluția: multiple readers writers problem.
- Intuitiv, dacă esti un reader ce intră în cameră, și vezi writers în cameră, iei loc (dacă esti o masină din est, și mașinile din vest traversează, aștepți).
- Dacă ești un reader, și sunt mai puțin de trei readers, și nu există writers în așteptare, intri în cameră.
- Dacă ești un reader, și sunt exact trei readers, aștepți.
- Dacă ești un reader ce părăsește camera, dacă nu mai sunt writers dar sunt readers în așteptare, permiți unui reader să intre.
- Dacă sunt writers în așteptare, și ești ultimul reader, permiți următorilor trei writers să intre.
- Principala diferență față de readers-writers e că vrem să permitem ca **exact** trei readers sau writers să intre simultan în cameră.

Invarianți

- Safety conditions: Readers și writers nu pot fi activi în același timp:
  - nr_active <= 3
  - nw_active <= 3
  - nr_active > 0 --> nw_active == 0
  - nw_active > 0 --> nr_active == 0
- Alte condiții:
  - nr_active > 0 & nr_waiting > 0 --> nw_waiting > 0
  - nw_active > 0 & nw_waiting > 0 --> nr_waiting > 0

Soluție pentru readers:

```
P( lock ); // lock is a mutex and is initializes to 1.
if (( nw_active + nw_waiting == 0 ) && ( nr_active < 3 )) {
     nr_active++; // notify we are active
     V( r_sem ); // allow ourself to get through
}  else
     nr_waiting++; // we are waiting
V( lock );
P( r_sem ); // readers will wait here, if they must wait.

READING...

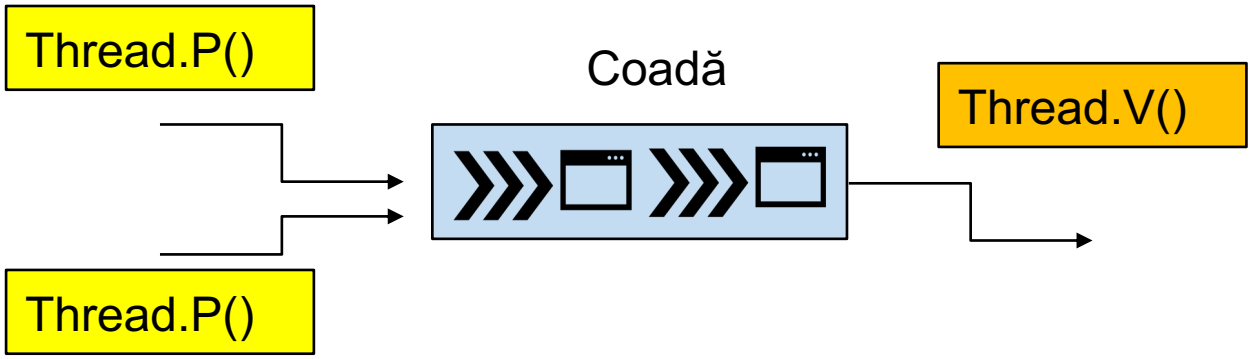
P( lock );
nr_active--;
if (( nr_active == 0 ) && ( nw_waiting > 0 ))  { // if we are the last reader
    count = 0; // makes sure we only let at most 3 writers in
    while (( nw_waiting > 0 ) && ( count < 3 )) {
          V( w_sem ); // wake a writer;
          w_active++; // one more active writer
          w_waiting--; // one less waiting writer.
          count++;
    }
}  else if (( nw_waiting == 0 ) && ( nr_waiting > 0 )) { // allow another waiting reader to go in, if no waiting writers
           V( r_sem );
           w_active++; // one more active reader
           w_waiting--; // one less waiting reader.
}
V( lock );
```

Initializați semafoarele

Să scriem soluția pentru writers...

Ordinea semafoarelor

- În momentul în care apelați V(), oricare thread blocat pe P() va fi trezit
  - Dar dacă doriți păstrarea ordinii FIFO?



Example: P and V

```
void v(struct semaphore *s) {
    struct proc_desc *p = 0;

    save = intr_enable( ALL_DISABLE );
    while ( TestAndSet( &s->sem_lock ) );
    s->sem_count++;
    if (p = get_from_queue( &s->sem_queue)) {
       p->runstate &= ~PROC_BLOCKED;
    }
    s->sem_lock = 0;
    intr_enable( save );
    if (p)
       reschedule(p);
}
```

```
void p(struct semaphore *s) {
    struct proc_desc *p = 0;
    for (;;) {
       save = intr_enable( ALL_DISABLE );
       while ( TestAndSet( &s->sem_lock ) );
       if (s->sem_count > 0) {
            s->sem_cont--;
            s->sem_lock = 0;
            intr_enable( save );
            return;
       }
       add_to_queue( &s->sem_queue, myproc );
       myproc->runstate |= PROC_BLOCKED;
       s->sem_lock = 0;
       intr_enable( save );
       yield();
    }
}
```

process A			**P**	**P**		**WAKE**
process B		**V**			**V**	
Semaphore						
lock	NO	YES	YES	YES	YES	YES
count	0	1	0		1	0
queue	0			A		
int disable	NO	YES	YES	YES	YES	YES

Semafor FIFO în Java

```
public class SemaphoreQueue
{
    private SemaphoreSlim semaphore;
    private ConcurrentQueue<TaskCompletionSource<bool>> queue =
        new ConcurrentQueue<TaskCompletionSource<bool>>();
    public SemaphoreQueue(int initialCount)
    {
        semaphore = new SemaphoreSlim(initialCount);
    }
    public SemaphoreQueue(int initialCount, int maxCount)
    {
        semaphore = new SemaphoreSlim(initialCount, maxCount);
    }
    public void Wait()
    {
        WaitAsync().Wait();
    }
    public Task WaitAsync()
    {
        var tcs = new TaskCompletionSource<bool>();
        queue.Enqueue(tcs);
        semaphore.WaitAsync().ContinueWith(t =>
        {
            TaskCompletionSource<bool> popped;
            if (queue.TryDequeue(out popped))
                popped.SetResult(true);
        });
        return tcs.Task;
    }
    public void Release()
    {
        semaphore.Release();
    }
}
```